United States International University-Africa

Ngong Deland Ngong - 670411

APT3065
Summer 2026
Project Documentation

ACADEMIC ADVISING APPOINTMENT AND CASE MANAGEMENT SYSTEM (AAACMS)

SYSTEM DESIGN & TECHNICAL REPORT

1. EXECUTIVE SUMMARY & PROBLEM STATEMENT

Traditional academic advising in higher education institutions frequently faces challenges such as scheduling conflicts, fragmented communication channels, lack of centralized record-keeping, and communication gaps between students and faculty advisors.

The Academic Advising Appointment and Case Management System (AAACMS) is a specialized web application developed to address these operational inefficiencies. Built with the Python/Django framework and Bootstrap 5, the platform centralizes appointment scheduling, enforces dynamic slot availability locking to prevent double-booking, and maintains an auditable history of advising case notes for each student.

2. SYSTEM OBJECTIVES & SCOPE

The primary objectives of the AAACMS include:

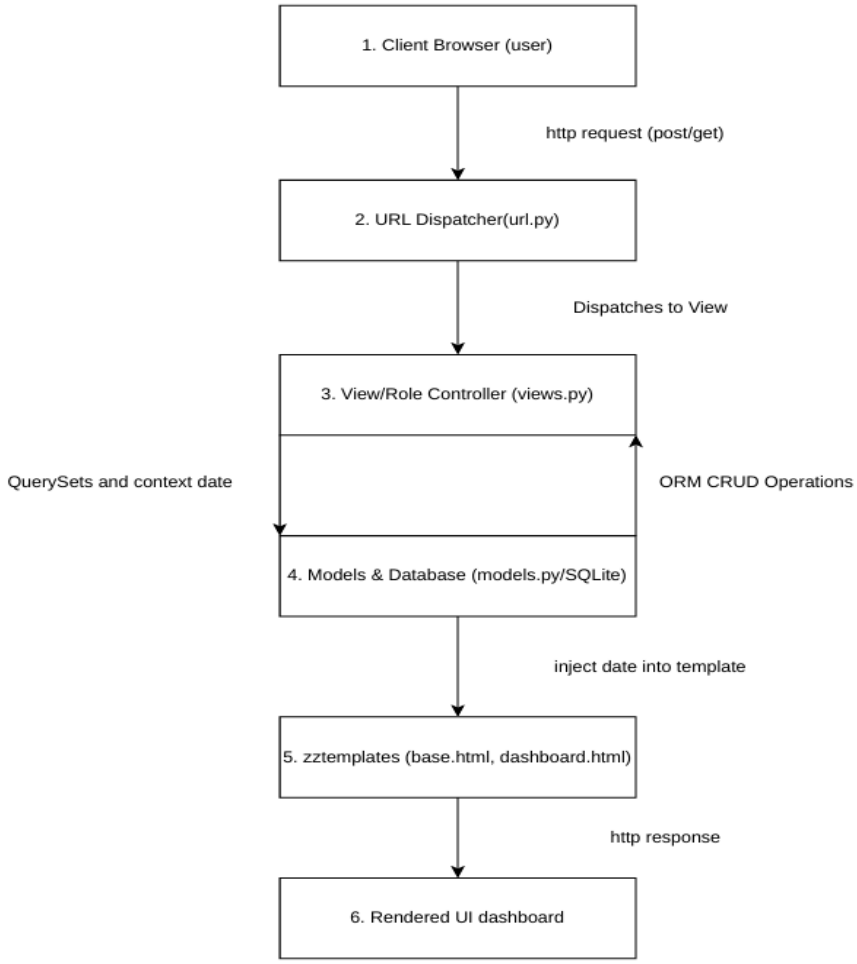
- Role-Based Access Control (RBAC): Differentiate system capabilities across three user roles: Students, Advisors, and Administrators.
- Real-Time Search & Discovery: Provide students with dynamic multi-field filtering to find open slots by advisor name or specific dates.
- Concurrency & Double-Booking Protection: Automatically update slot availability states upon booking to prevent concurrent double-booking.
- Institutional Case Management: Enable advisors to log academic recommendations, action items, and session notes directly linked to student records.
- User Feedback & UX Polish: Provide immediate visual feedback via Django flash messages and auto-dismissing Bootstrap alerts.

3. SYSTEM ARCHITECTURE & DESIGN PATTERNS

AAACMS utilizes Django's Model-View-Template (MVT) architectural pattern:

- Models (Data Layer): Encapsulates application data structures, entity relationships, data constraints, and business logic using Django's ORM.
- Views (Controller Layer): Handles incoming HTTP requests, session state, role-based routing, form processing, dynamic queryset filtering, and race-condition validation.
- Templates (Presentation Layer): Dynamic HTML5 templates powered by Django Template Language (DTL) and styled with Bootstrap 5 for responsiveness.
- Database: Relational SQLite database engine (with migration readiness for PostgreSQL or MySQL).

System Workflow Diagram



4. DATABASE SCHEMA & DATA DICTIONARY

The database schema consists of four interconnected relational models:

4.1 Model: User (Extended AbstractUser)

- username (CharField, PK/Unique): Primary identity handle for authentication.
- email (EmailField): Primary institutional email address.
- role (CharField): User classification ('student', 'advisor', 'admin').
- department (CharField, Optional): Academic or administrative department.

4.2 Model: AvailabilitySlot

- id (AutoField, PK): Primary key identifier.
- advisor (ForeignKey -> User): The advisor who published the slot.
- date (DateField): Date of the advising slot (YYYY-MM-DD).
- time_slot (CharField): Specific time range (e.g., "10:00 AM - 11:00 AM").
- is_booked (BooleanField, Default=False): State flag indicating if a slot is taken.

4.3 Model: Appointment

- id (AutoField, PK): Primary key identifier.
- student (ForeignKey -> User): Student who booked the session.
- advisor (ForeignKey -> User): Advisor assigned to the session.
- slot (ForeignKey -> AvailabilitySlot, Nullable): Associated slot record.
- date (DateField): Date of the session.
- time_slot (CharField): Scheduled time window.
- issue_category (CharField): Advising topic ('Academic Advising', 'Course Registration', 'Graduation Audit', 'Career Guidance', 'General Inquiry').
- appointment_mode (CharField): Delivery format ('In-Person', 'Virtual').
- meeting_link (URLField, Optional): Virtual meeting URL (e.g., Zoom, Google Meet).
- created_at (DateTimeField): Auto-generated timestamp.

4.4 Model: CaseLog

- id (AutoField, PK): Primary key identifier.
- appointment (ForeignKey -> Appointment): Linked advising appointment.
- advisor (ForeignKey -> User): Advisor authoring the case note.
- student (ForeignKey -> User): Student the session note pertains to.

- notes (TextField): Detailed notes, academic plans, and action items.
- created_at (DateTimeField): Auto-generated logging timestamp.

5. SECURITY & VALIDATION SPECIFICATIONS

- Cryptographic Hashing: User passwords are encrypted using PBKDF2 with SHA-256 and a unique cryptographic salt across all accounts.
- CSRF Protection: Django Cross-Site Request Forgery tokens ({% csrf_token %}) are enforced on all POST forms.
- Route Protection: Endpoints are secured with Django's @login_required decorator to prevent unauthorized URL access.
- Double-Booking Mitigation: The booking view performs an explicit validation check on `slot.is_booked` immediately prior to database commit, preventing race conditions if multiple users attempt booking simultaneously.
- Dynamic Queryset Restriction: Advisor case log forms are scoped dynamically so advisors can only create logs for appointments assigned to them.

6. USER ACCEPTANCE TESTING (UAT) & VERIFICATION

Test ID	Test Scenario	Expected Outcome	Result
TC-01	Student/Advisor Login	Routes user to role-based dashboard	PASS
TC-02	Publish Availability Slot	Slot created with is_booked=False	PASS
TC-03	Multi-Field Slot Search	Filters slots by Advisor or Date	PASS
TC-04	Schedule Appointment	Slot state updates to is_booked=True	PASS

TC-05	Double-Booking Attempt	Displays error flash alert	PASS
TC-06	Log Case Note	Notes saved and visible to student	PASS
TC-07	System Logout	Session clears with logout banner	PASS

7. DEPLOYMENT & INSTALLATION GUIDE

To run the AAACMS application locally:

1. Clone Repository & Navigate:

```
git clone <repository_url>
cd AAACMS_Project
```

2. Activate Virtual Environment:

```
# Windows:
env\Scripts\activate
# macOS/Linux:
source env/bin/activate
```

3. Install Required Dependencies:

```
pip install -r requirements.txt
```

4. Apply Database Migrations:

```
python manage.py makemigrations
python manage.py migrate
```

5. Create Administrative Superuser:

```
python manage.py createsuperuser
```

6. Launch Local Development Server:

```
python manage.py runserver
```

7. Access Portal:

- User Dashboard: http://127.0.0.1:8000/
- Django Admin: http://127.0.0.1:8000/admin/